20q_phi3_final_playable

June 14, 2026

1 20Q — Final (Entropy+Gini) with Optional Phi-3 Generator

Features: - Loads `property_universe.txt`, `noun_universe.txt`, and `all_per_student.json` from the notebook folder. - Optional Phi-3-mini-4k-instruct data generator (disabled by default). See the generator cell. - Hybrid entropy–Gini question selection and improved phrasing. - Runs 10 fixed-seed automated tests (simulated answers) and shows results in a DataFrame.

[2]:
```
# <a id="config"></a>
# Configuration & optional installs
import os, random
# Toggle: enable LLM-assisted generation (Phi-3). Default False.
USE_PHI3_GENERATOR = True

# Model name for generator (used only if USE_PHI3_GENERATOR=True)
MODEL_NAME = "microsoft/Phi-3-mini-4k-instruct"

# Filenames (assumed in the same folder as the notebook)
PROPERTY_FILE = "property_universe.txt"
NOUN_FILE = "noun_universe.txt"
ALL_PER_STUDENT = "all_per_student.json"

# Generated output file (written if generator runs)
GENERATED_OUTPUT = "generated_data.json"

# Fixed seeds for the 10 automated tests (we'll pick deterministic nouns later)
TEST_COUNT = 10
RANDOM_SEED = 12345  # base seed for reproducibility

# Install dependencies cell (toggleable)
# If you're running in Colab and want the notebook to auto-install missing␣
 ↪packages,
# uncomment and run the following block.
#
# !pip install -q transformers accelerate torch pandas matplotlib
#
# Note: Enable GPU in Colab runtime if you plan to run the Phi-3 generator.
```

[ ]:
```python
# <a id="utils"></a>
# Utility functions used across the notebook
import math, random, json, os, hashlib
from collections import Counter, defaultdict

def deterministic_bool(noun, prop, salt=0):
    """Deterministic pseudo-random boolean from noun+prop by hashing␣
 ↪(reproducible)."""
    h = hashlib.sha256(f"{salt}-{noun}-{prop}".encode('utf-8')).digest()
    return (h[0] % 2) == 0  # even->True, odd->False

def entropy(probs):
    s = 0.0
    for p in probs:
        if p > 0:
            s -= p * math.log2(p)
    return s

def gini(probs):
    return 1.0 - sum(p*p for p in probs)

def entropy_gini_score(true_count, false_count):
    total = true_count + false_count
    if total == 0:
        return 0.0
    p_true = true_count/total
    p_false = false_count/total
    # hybrid: weighted combination of entropy and gini
    return 0.6 * entropy([p_true, p_false]) + 0.4 * gini([p_true, p_false])
```

[ ]:
```python
# <a id="data"></a>
# Data loading that reads the three files in the notebook folder.
import json, os
from pprint import pprint

def read_list_file(path):
    with open(path, 'r', encoding='utf-8') as f:
        lines = [ln.strip() for ln in f if ln.strip()]
    return lines

def load_all_per_student(path):
    with open(path, 'r', encoding='utf-8') as f:
        data = json.load(f)
    return data

def build_universe_from_files(property_file=PROPERTY_FILE, noun_file=NOUN_FILE,␣
 ↪all_file=ALL_PER_STUDENT):
```

```
    # Read property and noun lists
    if not os.path.exists(property_file):
        raise FileNotFoundError(f"{property_file} not found in notebook folder.
 ↪")
    if not os.path.exists(noun_file):
        raise FileNotFoundError(f"{noun_file} not found in notebook folder.")
    if not os.path.exists(all_file):
        raise FileNotFoundError(f"{all_file} not found in notebook folder.")
    props = read_list_file(property_file)
    nouns = read_list_file(noun_file)
    all_per_student = load_all_per_student(all_file)
    return props, nouns, all_per_student

# Attempt to load; surface friendly message if missing
_loaded = False
try:
    PROPERTIES, NOUNS, ALL_PER_STUDENT_DATA = build_universe_from_files()
    print(f"Loaded {len(PROPERTIES)} properties and {len(NOUNS)} nouns from␣
 ↪files.")
    _loaded = True
except Exception as e:
    print("Warning: could not load your files automatically. Make sure the␣
 ↪three files are in the notebook folder:")
    print("  property_universe.txt, noun_universe.txt, all_per_student.json")
    print("Error:", e)
    PROPERTIES, NOUNS, ALL_PER_STUDENT_DATA = [], [], {}
```

```
Loaded 228 properties and 1852 nouns from files.
```

[ ]:
```
# <a id="phi3"></a>
# Optional Phi-3-mini-4k-instruct generator (preprocessing). Only runs if␣
 ↪USE_PHI3_GENERATOR=True
# NOTE: If you enable this in Colab, make sure you have enabled GPU in Runtime␣
 ↪-> Change runtime type -> GPU.
# This cell will attempt to load the model from transformers and run it; it can␣
 ↪be slow and requires sufficient RAM/VRAM.

def phi3_generate_properties(nouns, properties, outpath=GENERATED_OUTPUT,␣
 ↪append_if_exists=True, sample_rate=1.0):
    if not USE_PHI3_GENERATOR:
        print("Phi-3 generator is disabled. To enable, set USE_PHI3_GENERATOR =␣
 ↪True in the config cell.")
        return {}
    try:
        from transformers import AutoTokenizer, AutoModelForCausalLM
        import torch
```

```
    except Exception as e:
        raise RuntimeError("Transformers/torch not installed in environment. Install them to run generator.") from e

    print("Initializing model (this may take a while). Please ensure GPU runtime is enabled if available.")
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_NAME,
        device_map="cuda" if torch.cuda.is_available() else "cpu",
        dtype="auto",
        trust_remote_code=False,
    )

    generated = {}
    for noun in nouns:
        # sample_rate allows skipping some nouns if you want to generate fewer items
        if random.random() > sample_rate:
            continue
        props_for_noun = {}
        for prop in properties:
            # For each noun+property we ask model to produce an integer 1-10 (scale) and threshold at 6
            prompt = f"Rate the property '{prop}' for the noun '{noun}' with a single integer 1-10 (1=low,10=high). Respond with the integer only."
            inputs = tokenizer(prompt, return_tensors='pt').to(model.device)
            out = model.generate(**inputs, max_new_tokens=16, do_sample=False)
            txt = tokenizer.decode(out[0], skip_special_tokens=True)
            # extract first integer
            import re
            m = re.search(r"([1-9]|10)", txt)
            if m:
                val = int(m.group(0))
                props_for_noun[prop] = (val >= 6)  # threshold
            else:
                # fallback to deterministic bool if model didn't respond as expected
                props_for_noun[prop] = deterministic_bool(noun, prop, salt=42)
        generated[noun] = props_for_noun

    # write to disk (append or overwrite)
    if os.path.exists(outpath) and append_if_exists:
        try:
            with open(outpath, 'r', encoding='utf-8') as f:
                existing = json.load(f)
```

```
        except Exception:
            existing = {}
        existing.update(generated)
        to_write = existing
    else:
        to_write = generated
    with open(outpath, 'w', encoding='utf-8') as f:
        json.dump(to_write, f, indent=2)
    print(f"Wrote generated data for {len(generated)} nouns to {outpath}")
    return generated
```

[ ]:
```
# <a id="scoring"></a>
# Question scoring and phrasing utilities
import random

QUESTION_TEMPLATES = [
    "Would you say it is {}?",
    "Is it commonly described as {}?",
    "Does it usually have the property of being {}?",
    "Could it be described as {}?",
    "Is it often {}?"
]

def phrase_property(prop):
    # make grammar a bit better for common verbs/adjectives
    p = prop.replace('_', ' ')
    # simple heuristics: if startswith 'can ' or 'has ' keep as is, else use␣
 ↪article
    if p.startswith('can ') or p.startswith('has '):
        phr = p
    else:
        phr = p
    template = random.choice(QUESTION_TEMPLATES)
    return template.format(phr)

def choose_best_prop(records, asked):
    # records: list of dicts {'noun','properties':{prop:bool}}
    # compute hybrid score for each unused property
    scores = {}
    if not records:
        return None
    props = list(records[0]['properties'].keys())
    for prop in props:
        if prop in asked:
            continue
        true_count = sum(1 for r in records if r['properties'].get(prop, False))
        false_count = len(records) - true_count
```

```
        score = entropy_gini_score(true_count, false_count)
        # penalize unbalanced and near-constant properties slightly (already␣
 ↪handled in score)
        # also penalize properties that split out extremely few candidates
        if true_count == 0 or false_count == 0:
            score *= 0.0
        scores[prop] = score
    if not scores:
        return None
    # pick top score; break ties deterministically
    best = max(sorted(scores.items()), key=lambda x: x[1])
    return best[0]
```

[ ]:
```
# <a id="game"></a>
# Game logic for deterministic simulated play (uses ground truth prop map to␣
 ↪answer)
def make_records_from_map(nouns, prop_map, properties):
    records = []
    for n in nouns:
        props = {p: bool(prop_map.get(n, {}).get(p, False)) for p in properties}
        records.append({'noun': n, 'properties': props})
    return records

def simulate_game(target_noun, records, max_q=20, verbose=False):
    # Simulate a play where answers come from the target_noun's properties␣
 ↪(ground truth).
    records_copy = [dict(r) for r in records]
    remaining = [r for r in records_copy]
    asked = set()
    q_count = 0
    target_record = next(r for r in records if r['noun'] == target_noun)
    while q_count < max_q:
        # If only one candidate remains, guess
        if len(remaining) == 1:
            q_count += 1
            guess = remaining[0]['noun']
            correct = (guess == target_noun)
            if verbose:
                print(f"Q{q_count}: Are you thinking of {guess}? -> {'yes' if␣
 ↪correct else 'no'}")
            return {'target': target_noun, 'guessed': guess, 'questions':␣
 ↪q_count, 'correct': correct}
        # Check early confident guess: top candidate by simple ranking
        # ranking by number of matching properties with target
        ranking = sorted(remaining, key=lambda r: sum(1 for p,v in␣
 ↪r['properties'].items() if v and target_record['properties'].get(p)),␣
 ↪reverse=True)
```

```
        top = ranking[0]['noun']
        # if top is very likely (exact match on all true props of target),␣
↪guess early
        top_record = next(r for r in remaining if r['noun']==top)
        matches_with_target = sum(1 for p in top_record['properties'] if␣
↪top_record['properties'][p] == target_record['properties'].get(p))
        if matches_with_target >= len([p for p,v in target_record['properties'].
↪items() if v]) and len(remaining) <= 3:
            q_count += 1
            guess = top
            correct = (guess == target_noun)
            if verbose:
                print(f"Q{q_count}: Are you thinking of {guess}? -> {'yes' if␣
↪correct else 'no'}")
            return {'target': target_noun, 'guessed': guess, 'questions':␣
↪q_count, 'correct': correct}
        # Choose best property to ask
        prop = choose_best_prop(remaining, asked)
        if prop is None:
            # fallback to guess top candidate
            q_count += 1
            guess = ranking[0]['noun']
            correct = (guess == target_noun)
            if verbose:
                print(f"Q{q_count}: Are you thinking of {guess}? -> {'yes' if␣
↪correct else 'no'}")
            return {'target': target_noun, 'guessed': guess, 'questions':␣
↪q_count, 'correct': correct}
        # Ask property
        q_count += 1
        answer = target_record['properties'].get(prop, False)
        if verbose:
            print(f"Q{q_count}: {phrase_property(prop)} -> {'yes' if answer␣
↪else 'no'}")
        asked.add(prop)
        # Narrow remaining by answer
        remaining = [r for r in remaining if r['properties'].get(prop, False)␣
↪== answer]
        if not remaining:
            # recovery: if contradictions, reset remaining and continue (allows␣
↪some robustness)
            remaining = [r for r in records_copy if r['noun'] != top]  # remove␣
↪top guess that caused contradiction
    # If we exit loop without guessing, final random pick among remaining
    final_guess = random.choice(remaining)['noun'] if remaining else 'UNKNOWN'
```

```
    return {'target': target_noun, 'guessed': final_guess, 'questions': max_q,␣
 ↪'correct': final_guess==target_noun}
```

[ ]:
```
# <a id="tests"></a>
# Run 10 fixed-seed simulated tests and display results in a pandas DataFrame
import random, pandas as pd

def build_prop_map_from_files(PROPERTIES, NOUNS, all_per_student,␣
 ↪generated_data_path=GENERATED_OUTPUT):
    # Try to build prop_map from generated_data_path if exists; else create␣
 ↪deterministic boolean map
    prop_map = {}
    # If generated file exists, load and use it (it can be created by phi3␣
 ↪generator)
    if os.path.exists(generated_data_path):
        try:
            with open(generated_data_path, 'r', encoding='utf-8') as f:
                gen = json.load(f)
            for n,props in gen.items():
                prop_map[n] = {p: bool(v) for p,v in props.items()}
            print(f"Loaded generated properties from {generated_data_path}␣
 ↪({len(prop_map)} nouns)")
            return prop_map
        except Exception as e:
            print("Could not load generated_data.json, falling back to␣
 ↪deterministic mapping.", e)

    # Fallback deterministic mapping: use hash to produce reproducible booleans
    for n in NOUNS:
        prop_map[n] = {}
        for p in PROPERTIES:
            prop_map[n][p] = deterministic_bool(n, p, salt=42)
    print(f"Built deterministic property map for {len(prop_map)} nouns (no␣
 ↪generated file found)." )
    return prop_map

# Build prop_map
PROP_MAP = build_prop_map_from_files(PROPERTIES, NOUNS, ALL_PER_STUDENT_DATA,␣
 ↪GENERATED_OUTPUT)

# Prepare records used by simulator
ALL_RECORDS = make_records_from_map(NOUNS, PROP_MAP, PROPERTIES)

# Choose 10 nouns deterministically using base seed
random.seed(RANDOM_SEED)
if len(NOUNS) < TEST_COUNT:
```

```
    test_nouns = NOUNS[:]  # if fewer than 10, use all
else:
    # choose first TEST_COUNT nouns (deterministic) to make seeds stable and predictable
    test_nouns = NOUNS[:TEST_COUNT]

# Run simulations
results = []
for t in test_nouns:
    res = simulate_game(t, ALL_RECORDS, max_q=20, verbose=False)
    results.append(res)

# Display results in a DataFrame
df = pd.DataFrame(results)
df = df[['target','guessed','questions','correct']]
display(df)
acc = df['correct'].mean() * 100.0
avg_q = df['questions'].mean()
print(f"\nSummary: Accuracy = {acc:.1f}% | Average questions = {avg_q:.1f} over {len(df)} tests.")
```

```
Built deterministic property map for 1852 nouns (no generated file found).
```

	target	guessed	questions	correct
0	aardvark	aardvark	11	True
1	acai	acai	11	True
2	action figure	action figure	10	True
3	actor	actor	10	True
4	adobe	adobe	10	True
5	advanced micro	advanced micro	11	True
6	aerodactyl	aerodactyl	11	True
7	aeronautical engineer	aeronautical engineer	10	True
8	african elephant	african elephant	11	True
9	africatwin	africatwin	11	True

```
Summary: Accuracy = 100.0% | Average questions = 10.6 over 10 tests.
```

[ ]:
```
# <a id="cleanup"></a>
# Notes and cleanup helpers
print("Notebook completed. If you ran the Phi-3 generator, generated data is saved to 'generated_data.json'.")
print("If you want reproducible tests with different nouns, edit the TEST_COUNT and selection logic in the tests cell.")
```

```
Notebook completed. If you ran the Phi-3 generator, generated data is saved to
'generated_data.json'.
If you want reproducible tests with different nouns, edit the TEST_COUNT and
```

selection logic in the tests cell.

[ ]:
```
# Manual interactive play (Option B: user 'thinks' of a noun)
import pandas as pd, random

def play_manual(nouns=None, prop_map=None, properties=None):
    if nouns is None:
        nouns = globals().get('NOUNS', None)
    if prop_map is None:
        prop_map = globals().get('PROP_MAP', None)
    if properties is None:
        properties = globals().get('PROPERTIES', None)

    if nouns is None or properties is None:
        raise RuntimeError('NOUNS or PROPERTIES not loaded. Run the␣
↪data-loading cells first.')

    records = make_records_from_map(nouns, prop_map if prop_map is not None␣
↪else build_prop_map_from_files(PROPERTIES, NOUNS, ALL_PER_STUDENT_DATA,␣
↪GENERATED_OUTPUT), properties)
    remaining = records[:]
    asked = set()
    q_count = 0

    print("\nThink of a noun from the universe and answer yes/no. Type 'quit'␣
↪to exit early.\n")
    while q_count < 20 and len(remaining) > 1:
        prop = choose_best_prop(remaining, asked)
        if prop is None:
            print('No informative property left. Will guess from remaining␣
↪candidates.')
            break
        qtext = phrase_property(prop)
        ans = input(f"Q{q_count+1}: {qtext} (y/n): ").strip().lower()
        if ans in ['quit','q']:
            print('Exiting manual play.')
            return
        if ans not in ['y','n','yes','no']:
            print("Please answer 'y' or 'n'.")
            continue
        ans_bool = ans.startswith('y')
        asked.add(prop)
        q_count += 1
        remaining = [r for r in remaining if r['properties'].get(prop, False)␣
↪== ans_bool]
        if len(remaining) == 1:
            guess = remaining[0]['noun']
```

```
            final = input(f"Are you thinking of {guess}? (y/n): ").strip().
 ↪lower()
            if final.startswith('y'):
                print(f"\n  I guessed it! It was {guess}.\n")
                return
            else:
                remaining = [r for r in remaining if r['noun'] != guess]
                print('Okay, I will continue...')
                continue
    if remaining:
        candidates = [r['noun'] for r in remaining]
        print(f"\nI couldn't be certain. Top candidates: {candidates[:10]}")
    else:
        print('\nNo candidates remain. There may be inconsistent answers.')


def simulate_list(nouns_list, verbose=True, cap=10):
    results = []
    if len(nouns_list) > cap:
        print(f"Limiting to first {cap} nouns for safety.")
        nouns_list = nouns_list[:cap]

    # Ensure PROP_MAP and ALL_RECORDS exist
    if 'PROP_MAP' not in globals():
        globals()['PROP_MAP'] = build_prop_map_from_files(PROPERTIES, NOUNS,␣
 ↪ALL_PER_STUDENT_DATA, GENERATED_OUTPUT)
    if 'ALL_RECORDS' not in globals():
        globals()['ALL_RECORDS'] = make_records_from_map(NOUNS, PROP_MAP,␣
 ↪PROPERTIES)

    for target in nouns_list:
        if target not in [r['noun'] for r in ALL_RECORDS]:
            print(f"Warning: target '{target}' not found in noun universe;␣
 ↪skipping.")
            continue
        res = simulate_game(target, ALL_RECORDS, max_q=20, verbose=verbose)
        results.append(res)

    if results:
        import pandas as pd
        df = pd.DataFrame(results)[['target','guessed','questions','correct']].
 ↪rename(columns={
            'target':'Target','guessed':'Guessed','questions':
 ↪'Questions','correct':'Correct'
        })
        display(df)
```

```
        print(f"\nSummary: Accuracy = {df['Correct'].mean()*100:.1f}% | Average␣
 ↪questions = {df['Questions'].mean():.1f} over {len(df)} tests.")
        return df
    else:
        print('No results to display.')
        return None

# Example usage (editable):
try:
    example_targets = NOUNS[:5]
except Exception:
    example_targets = ['pilot','teacher','doctor','car','dog']
print('Example simulation for:', example_targets)
simulate_list(example_targets, verbose=True, cap=10)
```

```
Example simulation for: ['aardvark', 'acai', 'action figure', 'actor', 'adobe']
Q1: Could it be described as camouflage? -> yes
Q2: Would you say it is brightness? -> yes
Q3: Does it usually have the property of being aggressiveness? -> yes
Q4: Does it usually have the property of being appeal? -> no
Q5: Is it commonly described as breakability? -> yes
Q6: Does it usually have the property of being activity level? -> no
Q7: Is it often accessibility? -> yes
Q8: Could it be described as carbon-footprint? -> no
Q9: Is it commonly described as accessibility global? -> yes
Q10: Does it usually have the property of being accurate? -> no
Q11: Are you thinking of aardvark? -> yes
Q1: Would you say it is camouflage? -> yes
Q2: Could it be described as brightness? -> no
Q3: Does it usually have the property of being accuracy? -> no
Q4: Is it often biodiversity? -> yes
Q5: Is it commonly described as annual precipitation? -> no
Q6: Is it often aroma? -> yes
Q7: Is it often accessibility global? -> yes
Q8: Is it commonly described as accessibility? -> yes
Q9: Does it usually have the property of being aggressiveness? -> yes
Q10: Would you say it is activity level? -> yes
Q11: Are you thinking of acai? -> yes
Q1: Is it often camouflage? -> no
Q2: Could it be described as amount of shedding? -> no
Q3: Is it often aquatic adaptation? -> no
Q4: Is it often accuracy? -> yes
Q5: Is it commonly described as agility? -> no
Q6: Is it commonly described as annual precipitation? -> yes
Q7: Is it commonly described as accurate? -> yes
Q8: Would you say it is activity level? -> no
Q9: Is it commonly described as accessibility? -> no
```

```
Q10: Are you thinking of action figure? -> yes
Q1: Does it usually have the property of being camouflage? -> yes
Q2: Does it usually have the property of being brightness? -> yes
Q3: Would you say it is aggressiveness? -> yes
Q4: Could it be described as appeal? -> yes
Q5: Does it usually have the property of being biodiversity? -> yes
Q6: Would you say it is accessibility global? -> yes
Q7: Is it often amount of shedding? -> no
Q8: Could it be described as artificiality? -> no
Q9: Would you say it is accuracy? -> yes
Q10: Are you thinking of actor? -> yes
Q1: Would you say it is camouflage? -> no
Q2: Is it commonly described as amount of shedding? -> no
Q3: Is it often aquatic adaptation? -> no
Q4: Is it commonly described as accuracy? -> no
Q5: Does it usually have the property of being company age? -> yes
Q6: Would you say it is aggressiveness? -> yes
Q7: Could it be described as accessibility? -> no
Q8: Would you say it is appeal? -> yes
Q9: Is it often accurate? -> yes
Q10: Are you thinking of adobe? -> yes
```

	Target	Guessed	Questions	Correct
0	aardvark	aardvark	11	True
1	acai	acai	11	True
2	action figure	action figure	10	True
3	actor	actor	10	True
4	adobe	adobe	10	True

```
Summary: Accuracy = 100.0% | Average questions = 10.4 over 5 tests.
```

[ ]:

	Target	Guessed	Questions	Correct
0	aardvark	aardvark	11	True
1	acai	acai	11	True
2	action figure	action figure	10	True
3	actor	actor	10	True
4	adobe	adobe	10	True